# UNIT II

## Software Requirements and Project Planning

### Introduction

Software Requirement Engineering (SRE) is the systematic process of discovering, analyzing, documenting, and managing the requirements of a software system.
It is a critical phase in the Software Development Life Cycle (**SDLC**) because it lays the foundation for design, implementation, and testing.

### Objectives of Requirement Engineering

- To understand **what the customer needs**.
- To define **clear, complete, and unambiguous requirements**.
- To provide a **basis for system design and testing**.
- To manage changes to requirements during the project.
- To ensure the developed system is **acceptable and usable** by users.

### Types of Requirements

**(a) Functional Requirements:** These describe **what the system should do**. They define system behavior, features, and functions.

**Example:**

- The system shall allow users to log in using a username and password.
- The system shall generate monthly sales reports.

**(b) Non-Functional Requirements:** These describe **how the system performs** rather than what it does. They include quality attributes and constraints.

**Examples:**

- Performance (response time < 2 seconds)
- Security (data must be encrypted)

- Usability (user-friendly interface)
- Reliability and Availability
- Portability and Maintainability

**Requirement Engineering Process:** The process involves several structured steps to ensure quality requirements.

**Step 1: Requirement Elicitation:** Process of collecting requirements from stakeholders (clients, users, domain experts, etc.).

**Techniques:**

- Interviews
- Questionnaires
- Brainstorming
- Observation
- Document Analysis
- Prototyping
- Joint Application Development (JAD)

**Step 2: Requirement Analysis:** Involves examining, refining, and classifying gathered requirements .Main goals are to detect **conflicts, overlaps, and missing requirements.**

**Activities:**

- Feasibility Study
- Conflict Resolution
- Requirement Prioritization
- Modeling (DFD, UML diagrams, etc.)

**Step 3: Requirement Specification:** Documenting the analyzed requirements clearly and precisely in a **Software Requirement Specification (SRS)** document.

**SRS should be:**

- Correct
- Complete

- Consistent
- Unambiguous
- Verifiable
- Modifiable

**Step 4: Requirement Validation:** Ensures that the requirements accurately represent the user's needs.

**Techniques:**

- Reviews and Inspections
- Prototyping
- Test-case generation
- Requirement traceability

**Step 5: Requirement Management**

Maintaining and controlling changes in requirements during the software project lifecycle.

**Activities:**

- Version Control
- Change Tracking
- Impact Analysis
- Traceability Matrix Maintenance

**Software Requirement Specification (SRS)**

The **Software Requirement Specification (SRS)** is a **formal document** that describes in detail what the proposed software system must do. It acts as a **contract** between the **client (customer)** and the **developer (software team)** — defining *what to build* but not *how to build it*. In other words, the SRS specifies **the requirements of the system clearly, completely, and precisely** before actual development starts.

**Objectives of an SRS Document**

The main goals of preparing an SRS are:

1. To provide a **clear and complete description** of the system's functional and non-functional requirements.
2. To ensure **mutual understanding** between customer and developer.
3. To provide a **basis for system design, testing, and validation**.
4. To reduce the chances of **miscommunication and rework**.
5. To serve as a **reference and baseline** for future maintenance or enhancements.

**Characteristics of a Good SRS:** A well-written SRS should have the following properties:

| Characteristic | Description |
|---|---|
| **Correct:** | It must reflect the user's real needs. |
| **Complete:** | Includes all necessary requirements, inputs, outputs, and constraints. |
| **Unambiguous:** | Each requirement must have only one interpretation. |
| **Consistent:** | No conflicts among requirements. |
| **Verifiable:** | Each requirement should be testable or measurable. |
| **Feasible:** | Requirements must be implementable within available resources. |
| **Modifiable:** | Easy to update and revise as needed. |
| **Traceable:** | Each requirement should be traceable to its origin and related stages (design, code, and test). |

## Risk Management in Software Projects:

Risk Management in a software project is the process of identifying, analyzing, planning for, and controlling risks that may negatively affect the success of a software project. A risk is any uncertain event or condition that can have a positive or negative impact on project objectives like cost, schedule, quality, or scope.

### Objectives of Risk Management

1. To identify potential problems before they occur.
2. To minimize the impact of negative risks (threats).
3. To maximize the opportunities (positive risks).
4. To improve decision-making and project control.
5. To ensure successful completion of the software project within scope, time, and budget.

Risks can never be completely eliminated, but they can be **anticipated, reduced, and managed** using effective strategies. Two major approaches to managing risks are: Proactive Risk Management & Reactive Risk Management Strategy

**Proactive Risk Management Strategy:** **A** proactive strategy focuses on anticipating possible risks before they actually occur and taking preventive actions to reduce their likelihood or impact. It is a **forward-looking approach** — the project team actively **foresees problems**, **prepares mitigation plans**, and **avoids or minimizes risk effects** before they cause harm. In other words, the proactive approach asks:

"What could go wrong, and what can we do now to prevent it?"

### Objectives of Proactive Risk Management

- Identify potential risks early in the project.
- Develop strategies to avoid or minimize those risks.
- Enhance project preparedness.

- Reduce the uncertainty during project execution.
- Improve overall project control and confidence.

**Activities Involved in Proactive Strategy**

- **Risk Identification:** Predict possible risks related to technology, resources, people, schedule, or quality. Example: Risk of delay due to dependency on third-party software.
- **Risk Analysis:** Assess the **probability** and **impact** of each risk.
- Prioritize them using a **risk matrix** (High, Medium, Low).
- **Risk Prevention Planning:** Create **mitigation plans** or **preventive measures** to reduce the chance of risk occurrence. Example: Provide staff training to handle new technology.
- **Risk Monitoring:** Continuously review project progress to identify early warning signs.
- **Documentation:** Maintain a **Risk Register** with all identified risks, mitigation actions, and responsible persons.

**Reactive Risk Management Strategy**

A **reactive strategy** deals with **risks only after they have occurred**. It is a **response-based approach** — actions are taken **after** a problem appears, focusing on **damage control** and **recovery**.

In other words, the reactive approach asks: "What do we do now that the problem has happened?"

**Objectives of Reactive Risk Management**

- To minimize the impact of risks those have already occurred.
- To recover quickly and continue the project smoothly.
- To ensure business continuity despite disruptions.
- To learn from the incident and prevent it in future projects.

## Activities Involved in Reactive Strategy:

**Problem Detection:** Identify the risk or issue once it occurs (e.g., testing failure, server crash).
**Impact Assessment:** Evaluate how serious the problem is and which part of the project is affected.
**Corrective Actions:** Implement **contingency plans** or **emergency responses** to minimize damage.
**Project Adjustment:** Update schedule, budget, or scope to reflect the new situation.
**Post-Event Analysis:** Document the incident and response to improve future readiness.

## Risk Management Process:

1. **Risk Identification:** List all possible risks that may affect the project. Use techniques like: Brainstorming, Expert interviews, Checklists, Historical data analysis

**Example:** "Team may not be familiar with the new programming language."

2. **Risk Analysis (Assessment):** Determine **likelihood (probability)** and **impact (severity)** of each risk. Categorize risks as: High, Medium, Low

3. **Risk Prioritization:** Rank risks according to their potential effect on the project. Helps to focus on critical risks first. Often visualized using a **Risk Matrix**.

**Risk Response Planning: Develop strategies to deal with each major risk.**

**Response Strategies:**

- **Avoidance:** Change the plan to eliminate the risk.
  *Example:* Use proven technology instead of experimental one.
- **Mitigation:** Reduce the probability or impact.
  *Example:* Provide training to team members.

- **Transfer:** Shift the risk to a third party.
  *Example:* Outsource part of development to experts.
- **Acceptance:** Acknowledge the risk and prepare contingency plans.
  *Example:* Keep extra time or budget as buffer.

**5. Risk Monitoring and Control**

- Continuously track identified risks and identifies new ones.
- Review risk status during project meetings.
- Update the **Risk Register** regularly.

**Software Project Planning Fundamental:**

Software project planning is one of the **most crucial activities** in software engineering. It is the process of **defining, organizing, estimating, scheduling, and managing** all activities required to deliver a software product successfully.

Planning provides a **blueprint for action** — it guides the development team, aligns stakeholders' expectations, and ensures that the project is completed **on time, within budget, and with desired quality**.

Without effective planning, software projects may suffer from **delays, cost overruns, scope creep, or even failure**.

- **Objectives of Software Project Planning:** The main objectives of software project planning include:
- **Defining the Project Scope and Goals:** Clearly state what the project will deliver and what is out of scope.
- **Effort and Resource Estimation:** Estimate the manpower, tools, and time required.
- **Scheduling of Activities:** Prepare a detailed timeline with milestones and deliverables.
- **Cost Estimation and Budgeting:** Estimate project expenses and allocate budget accordingly.
- **Risk Management:** Identify possible risks and plan preventive and corrective measures.

- **Quality Planning:** Define quality assurance processes and metrics to ensure standards.
- **Communication and Coordination:** Define how information will flow between stakeholders and team members.
- **Project Control and Monitoring Plan:** Establish mechanisms to track progress and take corrective actions.

**The Software Project Planning Process:** The process of software project planning typically consists of several systematic steps. Each step builds upon the previous one to form a complete and executable plan.

- **Project Scope Definition : Scope** defines the **boundaries** of the project — what will be done, what deliverables will be produced, and what is outside the project.

It includes:

- Project objectives and deliverables
- Stakeholders and their roles
- Functional and non-functional requirements
- Constraints (time, budget, resources)

**Example:**
For a *Library Management System*, the scope may include:

- Book issue/return module
- Student registration
- Fine calculation
  But it may **exclude** online payment or mobile app integration.

- **Feasibility Study:** Before starting, it is necessary to check if the project is **feasible** from multiple perspectives.

| Type of Feasibility | Description |
|---|---|
| **Technical Feasibility** | Determines if technology and tools required are available. |
| **Economic Feasibility** | Determines if benefits outweigh costs. |
| **Operational Feasibility** | Checks whether the solution will be accepted and used by end users. |
| **Legal Feasibility** | Ensures compliance with laws and regulations. |

- **Effort Estimation:** Effort estimation predicts the **amount of work and manpower** required to complete the project. It helps in resource allocation and cost calculation.

**Common techniques:**

- **Expert Judgment:** Based on prior experience.
- **Analogous Estimation:** Based on similar past projects.
- **Algorithmic Models:** Mathematical models like ***COCOMO* (Constructive Cost Model)**.
- **Resource Planning:** After estimating effort, the next step is to identify resources needed:
- **Human Resources:** Developers, testers, analysts, designers, project managers.
- **Hardware/Software:** Servers, IDEs, tools, databases, testing frameworks.

- **Infrastructure:** Workstations, internet, office space, power backup.

A **Resource Allocation Matrix (RAM)** is often used to assign responsibilities (RACI – Responsible, Accountable, Consulted, and Informed).

**Scheduling:** Scheduling determines the **sequence and duration** of all project activities.
It answers: *"When will each task start and end?"*

**Tools and techniques used:**

- **Work Breakdown Structure (WBS)**: Breaks project into smaller tasks.
- **Gantt Chart:** Shows timeline of tasks and milestones.
- **PERT/CPM:** Identifies critical path (tasks that determine total project duration).

**Cost Estimation:** Cost estimation includes all expenses associated with the project:

- Manpower (salaries, overtime)
- Equipment and tools
- Software licenses
- Training
- Travel, maintenance, and overheads

Common models: *COCOMO* II, Delphi technique. A well-prepared budget ensures financial control.

**Risk Management Planning:** Every project faces risks that can affect schedule, cost, or quality. Risk management includes:

1. **Risk Identification** – List possible risks.

2. **Risk Analysis** – Assess likelihood and impact.
3. **Risk Prioritization** – Classify as High/Medium/Low.
4. **Risk Response Planning** – Avoid, Mitigate, Transfer, or Accept.
5. **Risk Monitoring** – Track risks continuously.

**Quality Planning:** Quality planning ensures the software will meet user requirements and standards. It includes:

- Defining **quality goals** and metrics.
- Planning **testing and review** activities.
- Setting up **process improvement** measures.
- Ensuring compliance with quality standards

**Communication Planning:** Defines **how information will be shared** within the team and with clients. It Includes:

- Meeting schedules and reporting formats
- Communication channels (email, chat, documentation tools)
- Stakeholder reporting frequency

**Project Plan Documentation:** All the above activities are documented in a **Software Project Plan (SPP)**, which includes:

- Project objectives and scope
- Work Breakdown Structure
- Schedule and milestones
- Cost and effort estimates
- Risk management plan
- Quality plan
- Communication and control procedures

The **SPP** serves as a **roadmap** and is approved before execution begins.

**Agile Planning and Estimation Techniques:**

Agile methodology focuses on **flexibility, adaptability, and continuous improvement** during software development. Unlike traditional approaches (like Waterfall), Agile allows teams to **respond**

**quickly to changes** in requirements and customer feedback.To achieve this, Agile teams rely on **effective planning and accurate estimation** — both of which are **iterative and adaptive** rather than rigid and fixed.

## What is Agile Planning?

Agile Planning is an iterative approach to planning software development that focuses on delivering small, usable pieces of the product in short time frames called iterations or sprints. It ensures that the team:

- Understands the work to be done,
- Prioritizes features based on customer value,
- Plans in short cycles (instead of entire project upfront), and
- Can adapt plans as requirements evolve.

## Key Principles of Agile Planning

1. **Iterative and Incremental** – Work is divided into small, manageable iterations.
2. **Customer Collaboration** – Customers are involved in planning and prioritization.
3. **Flexibility** – Plans can change as per feedback or new requirements.
4. **Value-driven** – High-value features are delivered first.
5. **Empowered Teams** – Teams decide how much work they can realistically commit to.

**Agile Estimation:** **Agile Estimation** is the process of estimating the effort, complexity, or time required to complete user stories or tasks in an Agile project.

Instead of traditional time-based estimation, Agile often uses relative estimation — comparing tasks against each other in terms of size, complexity, or effort.

## Objectives of Agile Estimation

- To help the team decide **how much work to commit** in an iteration.
- To support **prioritization** of backlog items.
- To enable better forecasting and release planning.
- To foster team collaboration and ownership.

## Agile Planning Process

The Agile planning process follows these main steps:

1. **Define Product Vision and Roadmap**
    - Understand overall product goals and features.
    - Create a high-level release plan.
2. **Create Product Backlog**
    - A list of all desired features, enhancements, and bug fixes.
    - Each feature is written as a User Story (e.g., *"As a student, I want to register online so that I can apply easily."*)
3. **Prioritize Backlog Items**
    - Using methods like **MoSCoW** (Must have, Should have, Could have, Won't have).
4. **Estimate User Stories**
    - Team estimates the effort or complexity of each story.
5. **Sprint Planning**
    - Team selects stories they can complete in the upcoming sprint.
6. **Execution and Review**
    - Daily scrums track progress.
    - Sprint reviews and retrospectives ensure continuous improvement.

**Agile Estimation Techniques:** There are several well-known techniques for Agile estimation. Here's a detailed explanation of each:

**Planning Poker:** A collaborative estimation game used by Agile teams to estimate the size of user stories.

**Steps:**

1. Each team member gets cards with numbers (e.g., 0, 1, 2, 3, 5, 8, 13, 21 – Fibonacci series).
2. The Product Owner reads a user story aloud.
3. Each member secretly selects a card that represents their estimate.
4. All cards are revealed simultaneously.
5. The team discusses differences and agrees on a final estimate.

**Benefits:**

- Encourages discussion
- Avoids bias
- Promotes team consensus

**Story Points:** A relative unit of measure used to express the complexity, effort, and uncertainty of a user story.

**Example:**

- Story A is estimated as **5 points**.
- Story B feels **twice as complex** → estimate as **10 points**.

Story points do not represent hours — they represent *relative effort*.

**Factors considered:**

- Complexity of the story
- Amount of work required
- Risks and dependencies

**T-Shirt Sizing:** A simple technique using **sizes** instead of numbers — such as XS, S, M, L, XL — to represent relative effort.

- "Login feature" → Small
- "Payment integration" → Large
- "Report generation" → Extra Large

**When to use:** In early project stages when details are unclear and For quick, high-level estimation.

**Tools for Agile Planning and Estimation**

| Tool | Purpose |
|---|---|
| **Jira** | Sprint planning, backlog management |
| **Trello** | Task tracking and visual boards |
| **VersionOne** | Agile project management |
| **Rally (CA Agile Central)** | Release and iteration planning |
| **Microsoft Azure DevOps** | Tracking progress and velocity |

**Benefits of Agile Planning and Estimation**

- Greater **flexibility and adaptability**
- Improved **customer satisfaction**
- Early and **frequent delivery** of working software
- Better **risk management** through short cycles
- Encourages **team collaboration and ownership**
- More **accurate forecasts** as project progresses

**Challenges in Agile Estimation:**

- Changing requirements
- Lack of experience in relative estimation
- Team size variation
- Overestimation or underestimation
- Pressure from stakeholders for fixed deadlines

**Important Question (Assignment: II)**

Q.1 What is software requirement engineering? Explain functional & non-functional requirements.

Q.2 Explain Software Requirement Specification (SRS)?

Q.3 Explain Software Requirement process.

Q.4 What is risk ? Explain risk management in software Project.

Q.5 Explain software project planning in detail.

Q.6 Explain agile planning and estimation techniques.